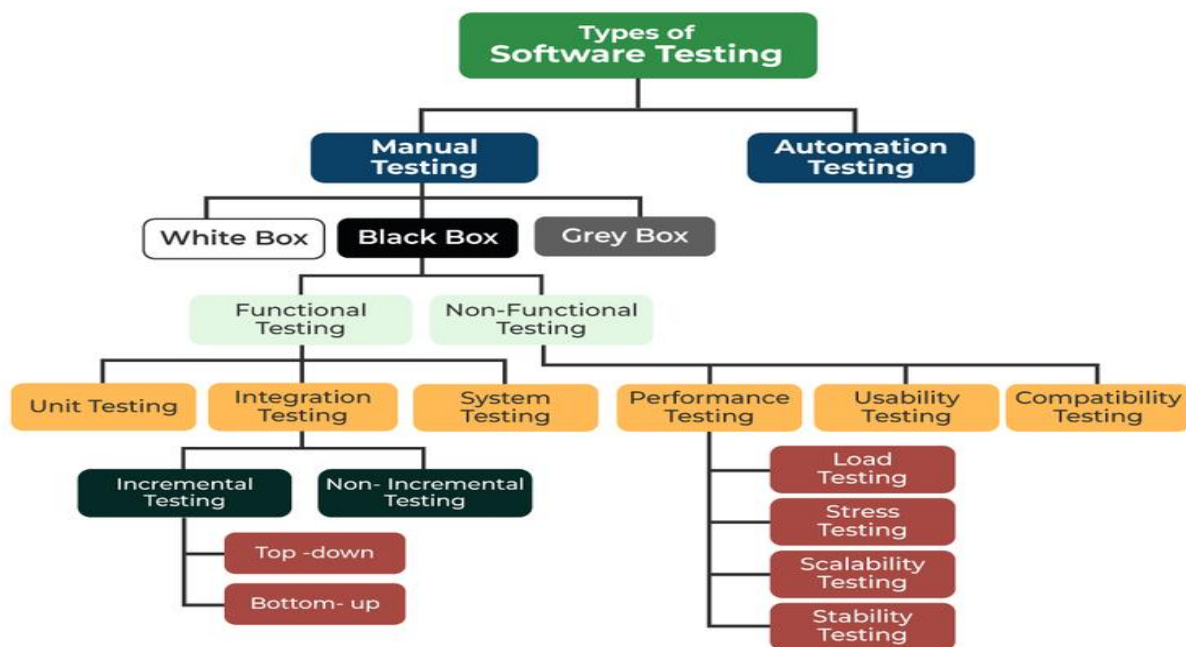


Software Testing:

Testing is the process of executing a program to find errors. To make our software perform well it should be error-free. If testing is done successfully it will remove all the errors from the software

[Software Testing](#) is a method to assess the functionality of the software program. The process checks whether the actual software matches the expected requirements and ensures the software is bug-free. The purpose of software testing is to identify the errors, faults, or missing requirements in contrast to actual requirements. It mainly aims at measuring the specification, functionality, and performance of a software program or application.



Types of Testing

- **Unit Testing:** Tests individual components or modules.
- **Integration Testing:** Tests interactions between integrated components.
- **System Testing:** Validates the entire system against requirements.
- **Acceptance Testing:** Ensures the software meets user needs and requirements.
- **Performance Testing:** Assesses system performance under varying conditions.
- **Security Testing:** Identifies vulnerabilities and ensures data protection.
- **Usability Testing:** Evaluates user-friendliness and interface effectiveness.

Software testing can be divided into two steps:

1. **Verification:** It refers to the set of tasks that ensure that the software correctly implements a specific function. It means “Are we building the product right?”

Verification is the process of checking that software achieves its goal without any bugs. It is the process to ensure whether the product that is developed is right or not. It verifies whether the developed product fulfills the requirements that we have. Verification is simply known as **Static Testing**.

Verification means checking the documents, languages, designs, and other programming things.

2. **Validation:** It refers to a different set of tasks that ensure that the software that has been built is traceable to customer requirements. It means “Are we building the right product?”

Validation is the process of checking whether the [software product](#) is up to the mark or in other words product has high-level requirements. It is the process of checking the validation of the product i.e. it checks what we are developing is the right product. it is a validation of actual and expected products. Validation is simply known as **Dynamic Testing**.

Validation means testing the actual product.

Verification	Validation
We check whether we are developing the right product or not.	We check whether the developed product is right.
Verification is also known as static testing .	Validation is also known as dynamic testing .
Verification includes different methods like Inspections, Reviews, and Walkthroughs.	Validation includes testing like functional testing , system testing, integration , and User acceptance testing.
It is a process of checking the work-products (not the final product) of a development cycle to decide whether the product meets the specified requirements.	It is a process of checking the software during or at the end of the development cycle to decide whether the software follow the specified business requirements.
Quality assurance comes under verification testing.	Quality control comes under validation testing.
The execution of code does not happen in the verification testing.	In validation testing, the execution of code happens.
In verification testing, we can find the bugs early in the development phase of the product.	In the validation testing, we can find those bugs, which are not caught in the verification process.
Verification testing is executed by the Quality assurance team to make sure that the product is developed according to customers' requirements.	Validation testing is executed by the testing team to test the application.
Verification is done before the validation testing.	After verification testing, validation testing takes place.
In this type of testing, we can verify that the inputs follow the outputs or not.	In this type of testing, we can validate that the user accepts the product or not.

Testing process:

The testing process in software engineering is a systematic approach designed to identify defects, ensure software quality, and validate that the software meets its specified requirements. This process typically includes various phases, each with specific objectives and activities. Here is a detailed overview

1. Requirement Analysis

Objective: Understand the requirements to determine the testing needs and scope.

Activities:

- Review and analyze requirement specifications.
- Identify the types of tests needed (functional, performance, security, etc.).
- Define test objectives and criteria for success.
- Identify testing priorities and critical areas.

2. Test Planning

Objective: Develop a test plan that outlines the testing strategy, scope, resources, schedule, and activities.

Activities:

- Define the scope and objectives of testing.
- Identify resources (personnel, tools, environments).
- Determine the testing schedule and milestones.
- Establish roles and responsibilities.
- Identify test deliverables (test plans, test cases, test scripts, reports).
- Conduct risk analysis and mitigation planning.

3. Test Case Development

Objective: Design and develop detailed test cases and scripts.

Activities:

- Create test cases based on requirement specifications.
- Develop test scripts for automated testing.
- Prepare test data required for test execution.

- Review and validate test cases and scripts to ensure they cover all aspects of the requirements.

4. Test Environment Setup

Objective: Set up the hardware and software environment where testing will be executed.

Activities:

- Configure the test environment to match production as closely as possible.
- Install necessary software, tools, and applications.
- Ensure the environment is stable and accessible.
- Validate the environment setup to ensure it is ready for testing.

5. Test Execution

Objective: Execute the test cases and scripts to validate the software.

Activities:

- Execute manual and automated test cases.
- Record the outcomes of each test case.
- Compare actual results with expected results.
- Log any defects or issues found during execution.
- Retest fixed defects to verify their resolution.

6. Defect Reporting and Tracking

Objective: Identify, log, and manage defects throughout the testing lifecycle.

Activities:

- Document defects with detailed information (steps to reproduce, severity, etc.).
- Use a defect tracking system to monitor the status of defects.
- Prioritize and assign defects to developers for resolution.
- Track defect lifecycle from identification to closure.
- Retest resolved defects to ensure they are fixed properly.

7. Test Reporting

Objective: Provide comprehensive reports on the testing activities, results, and defects.

Activities:

- Compile and summarize test execution results.
- Generate defect reports and status updates.
- Document test coverage and metrics (pass/fail rates, defect density, etc.).
- Provide insights and recommendations based on test findings.
- Communicate results to stakeholders.

8. Test Closure

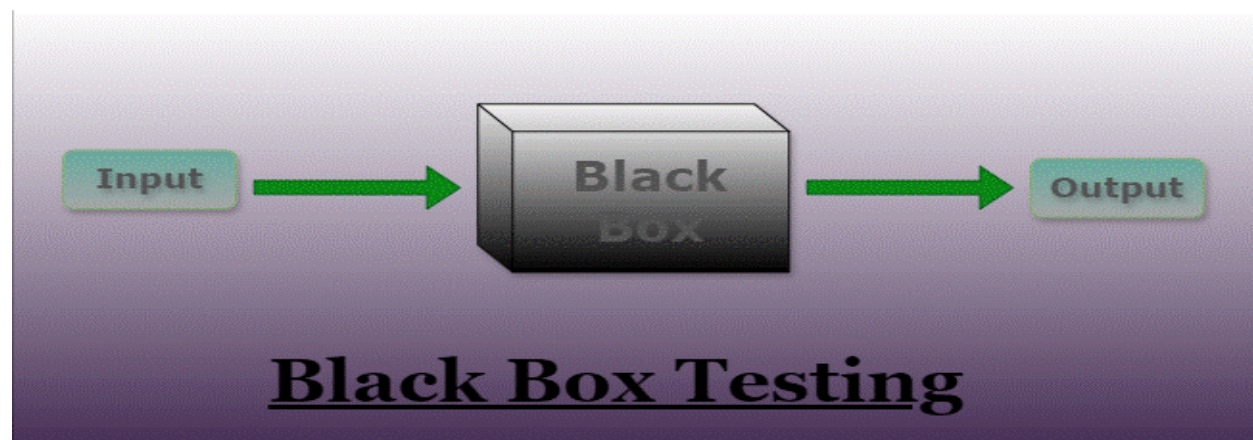
Objective: Conclude the testing activities and evaluate the process for future improvements.

Activities:

- Ensure all planned test cases have been executed.
- Confirm that all critical defects are resolved or documented.
- Archive test artifacts (test plans, cases, scripts, reports).
- Conduct a retrospective to identify lessons learned and areas for improvement.
- Provide a test closure report summarizing the testing process and results.

Black box or functional testing:

Black-box testing is a type of software testing in which the tester is not concerned with the internal knowledge or implementation details of the software but rather focuses on validating the functionality based on the provided specifications or requirements.



Types Of Black Box Testing

1. [Functional Testing](#)
2. [Regression Testing](#)
3. [Nonfunctional Testing \(NFT\)](#)

Functional Testing:

Functional testing is defined as a type of testing that verifies that each function of the software application works in conformance with the requirement and specification. This testing is not concerned with the source code of the application. Each functionality of the software application is tested by providing appropriate test input, expecting the output, and comparing the actual output with the expected output. This testing focuses on checking the user interface, APIs, database, security, client or server application, and functionality of the Application Under Test. Functional testing can be manual or automated. It determines the system's software functional requirements.

Advantages of Black Box Testing

- The tester does not need to have more functional knowledge or programming skills to implement the Black Box Testing.
- It is efficient for implementing the tests in the larger system.
- Tests are executed from the user's or client's point of view.
- Test cases are easily reproducible.
- It is used to find the ambiguity and contradictions in the functional specifications.

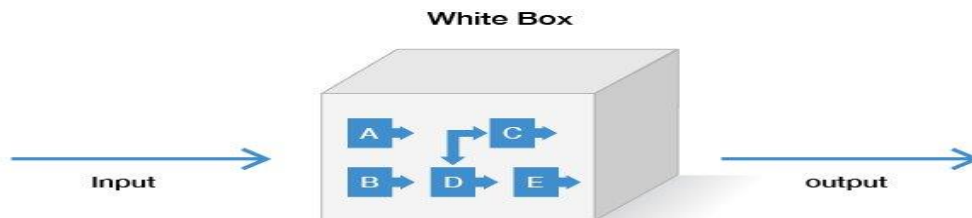
Disadvantages of Black Box Testing

- There is a possibility of repeating the same tests while implementing the testing process.
- Without clear functional specifications, test cases are difficult to implement.
- It is difficult to execute the test cases because of complex inputs at different stages of testing.
- Sometimes, the reason for the test failure cannot be detected.
- Some programs in the application are not tested.
- It does not reveal the errors in the control structure

White Box Testing or Structural testing

White Box Testing, or structural, code-based, or glass box testing, is a software testing technique that focuses on the software's internal logic, structure, and coding. It provides testers with complete application knowledge, including access to source code and design documents, enabling them to inspect and verify the software's inner workings, infrastructure, and integrations.

White box testing is also known as structural testing or code-based testing, and it is used to test the software's internal logic, flow, and structure. The tester creates test cases to examine the code paths and logic flows to ensure they meet the specified requirements

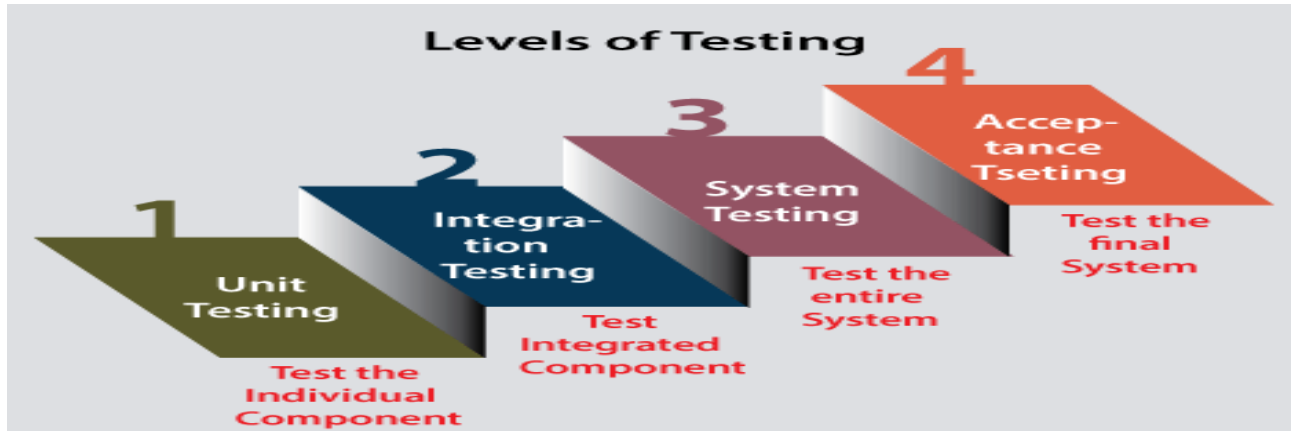


White Box vs. Black Box Testing

Aspect	White Box Testing	Black Box Testing
Access to Internal Details	Access to source code and design	No knowledge of internal code
Focus	Internal logic and structure	Functional behavior
Testing Level	Unit and integration testing	System and acceptance testing
Test Case Design	Based on code and logic	Based on requirements
Coverage Measurement	Statement, branch, path coverage	Equivalence, boundary checks
Objective	Uncover errors, vulnerabilities	Validate against requirements
Test Techniques	Code reviews, path coverage	Equivalence partitioning, etc.
Applicability	Programming errors, optimization	User functionality, behavior

Levels of testing: unit, integration, system, user acceptance testing

The levels of software testing involve the different methodologies, which can be used while we are performing the software testing.



Level1: Unit Testing

Unit testing is the first level of software testing, which is used to test if software modules are satisfying the given requirement or not.

The first level of testing involves **analyzing each unit or an individual component** of the software application.

Unit testing is also the first level of **functional testing**. The primary purpose of executing unit testing is to validate unit components with their performance.

Level2: Integration Testing

The second level of software testing is the **integration testing**. The integration testing process comes after **unit testing**.

It is mainly used to test the **data flow from one module or component to other modules**.

In integration testing, the **test engineer** tests the units or separate components or modules of the software in a group.

The primary purpose of executing the integration testing is to identify the defects at the interaction between integrated components or units.

Level3: System Testing

The third level of software testing is **system testing**, which is used to test the software's functional and non-functional requirements.

It is **end-to-end testing** where the testing environment is parallel to the production environment. In the third level of software testing, **we will test the application as a whole system.**

To check the end-to-end flow of an application or the software as a user is known as **System testing.**

Level4: Acceptance Testing

The **last and fourth level** of software testing is **acceptance testing**, which is used to evaluate whether a specification or the requirements are met as per its delivery.

The software has passed through three testing levels (**Unit Testing, Integration Testing, System Testing**). Some minor errors can still be identified when the end-user uses the system in the actual scenario.

In simple words, we can say that Acceptance testing is the **squeezing of all the testing processes that are previously done.**

The acceptance testing is also known as **User acceptance testing (UAT)** and is done by the customer before accepting the final product.

Tools and Techniques

- **Test Management Tools:** Jira, TestRail, Quality Center.
- **Automated Testing Tools:** Selenium, QTP/UFT, LoadRunner.
- **Defect Tracking Tools:** Bugzilla, JIRA, Mantis.
- **Performance Testing Tools:** JMeter, LoadRunner, NeoLoad.
- **Security Testing Tools:** OWASP ZAP, Burp Suite.